

When Do Simple Sorting Algorithms Outperform Advanced Ones?

An Experimental Study Across Different Input Types and Sizes

Mohammad Gayada
West University of Timișoara
`mohamad.ghayada04@e-uvt.ro`

May 2, 2026

Abstract

Sorting algorithms are one of those topics where theory and practice do not always agree. On paper, an algorithm with $O(n \log n)$ complexity should always beat one with $O(n^2)$, but in reality things are more nuanced than that. This paper presents an experimental comparison of twelve sorting algorithms tested across six different input types and three input sizes. The algorithms tested are Bubble Sort, Insertion Sort, Selection Sort, Gnome Sort, Cocktail Sort, Shell Sort, Merge Sort, Quick Sort, Heap Sort, Counting Sort, Radix Sort, and Bucket Sort.

The results show that for small inputs, simple algorithms are perfectly fine and sometimes even a bit faster. As input size grows, the efficient algorithms pull ahead quickly. Non-comparison algorithms like Counting Sort and Radix Sort were the fastest overall at $n = 1000$. The takeaway is that choosing an algorithm should depend on the actual data you are working with, not just what the textbook says. All code and raw results are available on GitHub (see Appendix A).

Contents

1	Introduction	3
1.1	Motivation	3
1.2	What I Did	3
1.3	A Small Example	3

1.4	What This Paper Contributes	4
1.5	Paper Structure	4
2	Algorithms and Their Complexity	4
2.1	Simple ($O(n^2)$) Algorithms	4
2.2	Efficient ($O(n \log n)$) Algorithms	5
2.3	Linear-Time (Non-Comparison) Algorithms	6
3	Implementation and Experimental Setup	6
3.1	Implementation	6
3.2	How Timing Was Done	7
3.3	Hardware and Environment	7
3.4	Input Types	7
3.5	Input Sizes and Exclusions	8
4	Results	8
4.1	Small Input ($n = 10$)	8
4.2	Medium Input ($n = 1000$)	8
4.3	Large Input ($n = 10000$)	9
4.4	How Input Type Affected the Results	9
5	Related Work	10
6	Conclusions and Future Work	11
6.1	What I Found	11
6.2	Difficulties Encountered	11
6.3	Future Work	12
A	Repository and Reproducibility	13

1 Introduction

1.1 Motivation

Sorting is something that comes up constantly in computer science, whether it is ordering database records, preparing data for binary search, or just cleaning up output before displaying it to a user [2]. Because of how common it is, a lot of research has gone into figuring out which sorting algorithms are the best. The standard answer, as taught in most algorithms courses and covered in textbooks like Cormen et al. [1], is that $O(n \log n)$ algorithms such as Merge Sort and Heap Sort are better than $O(n^2)$ ones like Bubble Sort or Insertion Sort.

That is true in general, but the question that motivated this paper is: when does that actually start to matter? Theoretical complexity describes how an algorithm scales, but it ignores constant factors, the cost of function calls in recursive algorithms, and how the shape of the input affects performance. For a small array, a simple algorithm might finish before a more sophisticated one even finishes setting itself up. And for data that is already almost sorted, an algorithm like Insertion Sort can be nearly linear, while something like Quick Sort might still do a lot of unnecessary work [3].

Most comparisons I could find online test only a few algorithms, or only random input, or do not say much about what happens at different input sizes. That is what this paper tries to address: a more complete picture across twelve algorithms, six input types, and three sizes.

1.2 What I Did

The idea is simple. All twelve algorithms were implemented in Java using the same interface, so they could be tested under identical conditions. Different kinds of input arrays were generated (random, sorted, reverse sorted, almost sorted, half sorted, and arrays with very few unique values), and each algorithm was run on each input type multiple times. The average execution time was recorded and compared.

The goal was not to find a single winner, but to understand under what conditions each algorithm performs well and when it starts to struggle.

1.3 A Small Example

To make this concrete, consider the array $[1, 2, 4, 3, 5, 6, 8, 7, 9, 10]$, which is sorted except for two swapped pairs. Insertion Sort will scan through the array and only need to move two elements, finishing almost instantly. Merge Sort will still split the array recursively to depth $\log_2 10 \approx 3$, merge everything back together, and only then produce the result.

At ten elements, neither one is noticeably slow, but the example shows why the structure of the input matters even at small sizes. Now imagine the same situation with a million elements.

1.4 What This Paper Contributes

To be clear about what is original here:

- A unified Java benchmarking framework for twelve sorting algorithms, made publicly available on GitHub.
- Timing data across six input distributions and three input sizes, covering a wider range of conditions than most informal comparisons.
- An analysis of when the crossover happens between simple and efficient algorithms.
- Observations about how specific input types affect specific algorithms in ways that are not obvious from the theory alone.

1.5 Paper Structure

Section 2 describes each algorithm and its complexity. Section 3 explains the implementation and how the experiments were set up. Section 4 goes through the results. Section 5 compares this work to related studies. Section 6 wraps things up and suggests directions for future work.

2 Algorithms and Their Complexity

This section gives a description of each algorithm and its complexity bounds. The proofs behind these bounds are not reproduced here; they can be found in Cormen et al. [1] and Knuth [2], which are the standard references for this material.

2.1 Simple ($O(n^2)$) Algorithms

Bubble Sort. Bubble Sort goes through the array repeatedly, comparing adjacent elements and swapping them if they are in the wrong order. Each full pass moves the largest unsorted element to its correct position. In the worst case — a reverse-sorted array — it needs $\Theta(n^2)$ comparisons. One small upside: if a pass completes with no swaps, the algorithm knows the array is already sorted and stops early, giving $\Theta(n)$ behaviour on already-sorted input.

Insertion Sort. Insertion Sort works like sorting a hand of playing cards. It picks one element at a time and shifts it left until it lands in the right place. In the worst case it is $\Theta(n^2)$, but for nearly sorted data it is much faster: the number of shifts is proportional to the number of inversions in the array, so it can run in $O(n + k)$ where k counts how many pairs are out of order [1]. This makes it surprisingly good on real-world data that is mostly ordered.

Selection Sort. Selection Sort repeatedly finds the smallest remaining element and swaps it to the front. Unlike the others, it always does exactly $\frac{n(n-1)}{2}$ comparisons no matter what the input looks like. Sorted, random, or reverse sorted — it does the same amount of work. Time complexity is $\Theta(n^2)$ with no exceptions.

Gnome Sort. Gnome Sort is essentially Insertion Sort written differently. Instead of shifting elements directly, it swaps the current element with its predecessor one step at a time, walking backward until the element is in place. It is not faster, just a different way of thinking about the same idea. Worst-case is $\Theta(n^2)$, and like Insertion Sort it handles nearly sorted data reasonably well.

Cocktail Sort. Cocktail Sort is a small improvement on Bubble Sort. Instead of only going left-to-right, it alternates direction: one pass forward, one pass backward. This helps avoid a known weakness of Bubble Sort where small values near the end of the array take many passes to reach their correct position. Despite this improvement, the worst-case complexity is still $\Theta(n^2)$.

2.2 Efficient ($O(n \log n)$) Algorithms

Shell Sort. Shell Sort extends Insertion Sort by first sorting elements that are far apart, then progressively narrowing the gap until it finishes with a regular Insertion Sort pass. The idea is to reduce the number of inversions quickly early on, so the final pass has very little to do. The complexity depends on the gap sequence used. With the sequence suggested by Knuth ($h = 3h + 1$) it is empirically close to $O(n \log^2 n)$ [3].

Merge Sort. Merge Sort splits the array in half, recursively sorts each half, and then merges the two sorted halves into one. The merge step is linear, and the recursion depth is $\log n$, giving $\Theta(n \log n)$ in all cases. It is stable and predictable, but it does need $O(n)$ extra memory for the merge step [1].

Quick Sort. Quick Sort picks a pivot, puts all smaller elements to the left and larger ones to the right, and then recurses on each side. On average this gives $\Theta(n \log n)$, but if

the pivot consistently splits the array badly — which happens with a fixed-last-element pivot on already-sorted input — the worst case is $\Theta(n^2)$. Our implementation uses the last element as the pivot, which is simple but has this known weakness on sorted data.

Heap Sort. Heap Sort builds a max-heap from the array in $O(n)$ time and then repeatedly extracts the largest element, shrinking the heap each time. This always runs in $\Theta(n \log n)$ and does not need any extra memory, unlike Merge Sort. It is not quite as fast in practice as Quick Sort on random data due to worse cache behaviour, but its worst case is guaranteed [1].

2.3 Linear-Time (Non-Comparison) Algorithms

There is a theoretical lower bound of $\Omega(n \log n)$ for any comparison-based sorting algorithm [1]. The algorithms below get around this by not comparing elements to each other directly, instead using the structure of the values themselves. They only work well under certain conditions.

Counting Sort. Counting Sort counts how many times each value appears, computes prefix sums to find where each value should go in the output, and then writes each element to its correct position. It runs in $\Theta(n + k)$ where k is the range of values. If k is much larger than n , it becomes impractical.

Radix Sort. Radix Sort processes numbers one digit at a time from least significant to most significant, using a stable sort (Counting Sort) at each step. For integers with a fixed number of digits the total time is $\Theta(d(n + k))$ where d is the number of digits and k is the digit base. For fixed-width integers this is effectively $\Theta(n)$ [1].

Bucket Sort. Bucket Sort divides elements into n buckets based on their value, sorts each bucket separately (usually with Insertion Sort), and concatenates the results. When the input is roughly uniformly distributed the average case is $\Theta(n)$, but if many elements fall into the same bucket it degrades toward $\Theta(n^2)$.

3 Implementation and Experimental Setup

3.1 Implementation

All twelve algorithms were written in Java 17. Each one implements a common `Sorter` interface with a single `sort(int[] arr)` method. This made it easy to plug any algorithm

into the same benchmarking loop without changing anything else. The full source is on GitHub (see Appendix A).

3.2 How Timing Was Done

Execution time was measured using `System.nanoTime()`, called immediately before and after the sort call. One thing noticed early on was that the first few runs of any algorithm were noticeably slower than the rest, probably because the JVM was still compiling the bytecode to native instructions (JIT compilation). To deal with this, each algorithm was run five times as a warm-up before any measurements were taken, and those runs were discarded.

After the warm-up, each (algorithm, input type, size) combination was run **20 times** and the arithmetic mean was recorded. Twenty repetitions was enough to smooth out noise from garbage collection and OS scheduling without making the total experiment time unreasonably long.

3.3 Hardware and Environment

The experiments were run on a laptop with an Intel Core i5-1135G7 processor (4 cores, 2.4 GHz base clock) and 8 GB of RAM, running Windows 11 and OpenJDK 17.0.9. Nothing else was running in the background during measurement.

3.4 Input Types

Six types of input were used:

1. **Random** — integers drawn uniformly from $[0, 10n]$.
2. **Sorted** — integers in ascending order.
3. **Reverse sorted** — integers in descending order.
4. **Almost sorted** — a sorted array with $\lfloor n/10 \rfloor$ random adjacent swaps applied.
5. **Half sorted** — the first half is sorted, the second half is random.
6. **Few unique elements** — integers drawn from $\{0, 1, \dots, 9\}$, so there are many duplicates.

These were chosen to cover a range of realistic scenarios. Random data represents the average case. Sorted and almost-sorted data reflect situations where some ordering already exists, like a log file that is roughly ordered by timestamp. Reverse sorted is

the classic worst case for several algorithms. Few unique elements tests how algorithms handle duplicates.

3.5 Input Sizes and Exclusions

Three sizes were tested: $n = 10$, $n = 1000$, and $n = 10000$.

For $n = 10000$, Bubble Sort, Selection Sort, Gnome Sort, and Cocktail Sort were left out. Each of these took several seconds per trial, which made 20 repetitions unreasonable in terms of total time. Their $O(n^2)$ complexity makes the outcome predictable anyway. Insertion Sort was kept at this size because it is fast enough on several input types to still be informative.

4 Results

4.1 Small Input ($n = 10$)

At ten elements, every algorithm finished in under a microsecond. The differences between them were within measurement noise and changed from run to run. Simple algorithms were occasionally slightly faster, likely because they avoid the overhead of recursive calls, but nothing was consistent enough to draw conclusions from. The main takeaway here is that for very small inputs, it genuinely does not matter which algorithm you use.

4.2 Medium Input ($n = 1000$)

This is where things started to get interesting. Table 1 shows the average execution time across all input types at $n = 1000$.

Counting Sort was the fastest, followed by Radix Sort — both benefiting from their linear-time complexity. Among comparison-based algorithms, Heap Sort and Quick Sort were the best, with Shell Sort close behind. Bubble Sort was the worst of the simple group, about ten times slower than Shell Sort.

Insertion Sort stood out among the $O(n^2)$ algorithms. It was significantly faster than the others, mostly because of how it performs on structured input. On almost-sorted arrays it barely had to do anything. Its average was pulled down by the random and reverse-sorted cases, but even then it was better than Bubble Sort or Cocktail Sort.

Algorithm	Avg. Time (s)
Bubble Sort	0.002228
Insertion Sort	0.000875
Selection Sort	0.001442
Cocktail Sort	0.001895
Gnome Sort	0.001557
Shell Sort	0.000248
Merge Sort	0.000374
Quick Sort	0.000292
Heap Sort	0.000277
Counting Sort	0.000113
Radix Sort	0.000179
Bucket Sort	0.001163

Table 1: Average execution time for $n = 1000$ across all input types.

4.3 Large Input ($n = 10000$)

At $n = 10000$ the gap between simple and efficient algorithms became very clear. Insertion Sort averaged around 0.019 seconds on random input, while Merge Sort stayed around 0.001 seconds — roughly 20 times faster. The other efficient algorithms (Heap Sort, Quick Sort, Shell Sort) all stayed under 0.002 seconds.

Quick Sort had an interesting result on sorted input. Because the implementation always picks the last element as the pivot, a sorted array causes worst-case $\Theta(n^2)$ behaviour: every partition is maximally unbalanced. On that input type at $n = 10000$, Quick Sort was considerably slower than usual, actually approaching Insertion Sort’s runtime. This is a well-known issue with naive Quick Sort implementations [3].

Counting Sort and Radix Sort remained the fastest overall, as expected from their linear-time complexity.

Figure 1 shows how execution time grows across the three sizes. The log scale makes it easier to compare algorithms with very different runtimes.

4.4 How Input Type Affected the Results

Input type made a large difference, especially for the simple algorithms. Insertion Sort on sorted input was roughly $50\times$ faster than on reverse-sorted input at $n = 1000$. That is a bigger gap than the one between Insertion Sort and Merge Sort on random data at the same size, which was a somewhat surprising finding.

Bucket Sort performed well on random and uniform input but degraded significantly on the “few unique elements” case, because almost everything ended up in a small number of buckets. The inner Insertion Sort then had to deal with a large portion of the array

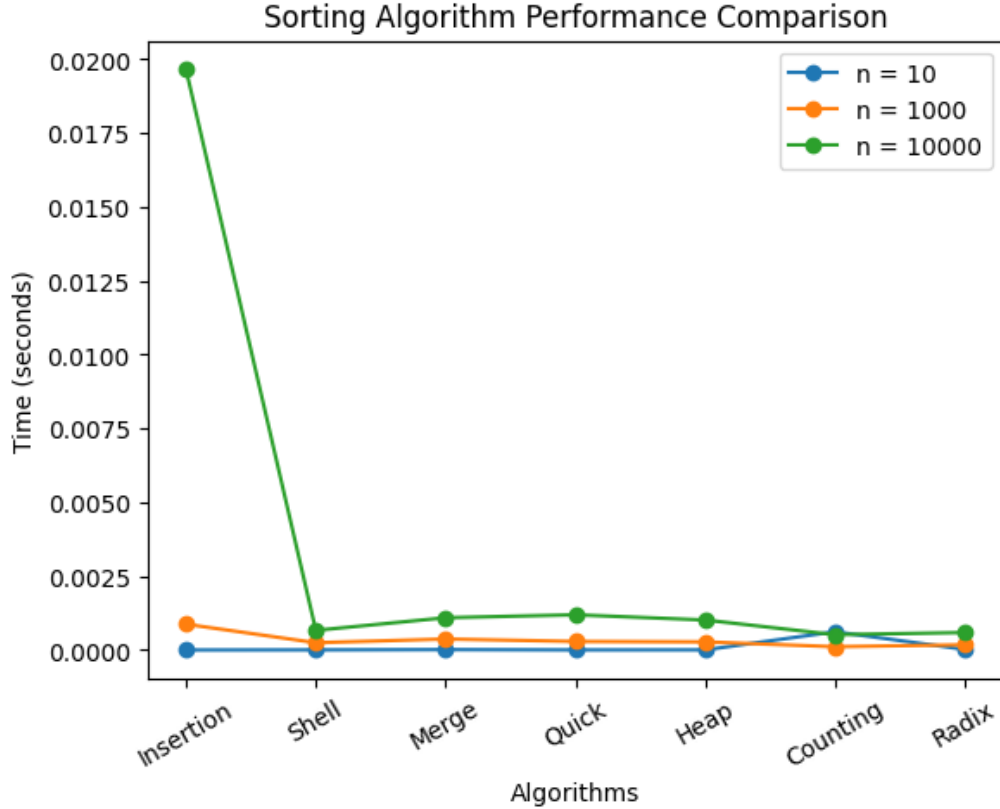


Figure 1: Execution time across input sizes (log scale). Simple algorithms grow steeply between $n = 1000$ and $n = 10000$, while efficient algorithms remain relatively flat.

all at once, effectively turning the whole thing into an $O(n^2)$ algorithm.

Selection Sort was completely unaffected by input type, which is both its strength and its weakness: you always know exactly how long it will take, but you cannot take advantage of any existing order in the data.

5 Related Work

Sorting algorithms are one of the most studied topics in computer science, so there is a lot of existing work to compare against. Knuth’s *The Art of Computer Programming* [2] is the foundational reference for most classical sorting algorithms and includes detailed mathematical analyses. Sedgewick and Wayne [3] cover similar material with more of a focus on practical implementation, including the advice of switching to Insertion Sort for small sub-arrays inside recursive algorithms — something that Java’s own standard library actually does.

There is also research on sorting in parallel and on GPUs, which goes in a different direction from this paper. Hirschberg [4] studied parallel comparison-based sorting with logarithmic depth. Satish et al. [5] developed highly optimised sorting for many-core

GPUs, achieving throughputs far beyond what any sequential algorithm can reach. Those results are not really comparable to what is done here, since this paper focuses on simple sequential in-memory sorting rather than high-performance computing.

The Java standard library uses a dual-pivot Quick Sort for primitive types and Tim Sort (a hybrid of Merge Sort and Insertion Sort) for object arrays. Both are more sophisticated than the implementations in this paper. The point of keeping things simple was not to compete with the standard library, but to make the behaviour of each algorithm transparent and easy to compare.

6 Conclusions and Future Work

6.1 What I Found

Going into this project, the results were roughly what was expected, but some things were more pronounced than anticipated. The main findings are:

1. At $n = 10$, none of the differences between algorithms are meaningful. Simple algorithms are perfectly adequate for small inputs.
2. At $n = 1000$, efficient algorithms are already noticeably faster — roughly 4 to 20 times so depending on the algorithm and input type. Non-comparison algorithms are faster still.
3. At $n = 10000$, the $O(n^2)$ algorithms are simply not practical for general use. The roughly $20\times$ gap between Insertion Sort and Merge Sort on random input speaks for itself.
4. Input type matters a lot, sometimes as much as algorithm choice. Insertion Sort on almost-sorted data beat Quick Sort on sorted data with a bad pivot, which is not something you would predict from complexity analysis alone.

6.2 Difficulties Encountered

The biggest issue during the experiments was JVM warm-up. Before adding the warm-up runs, the first few trials of every algorithm were much slower than the rest, which was throwing off the averages. Once five warm-up runs were added and discarded before measurement, the results became much more consistent.

Excluding the very slow algorithms at $n = 10000$ was also a practical necessity. It would have been better for completeness to include them, but waiting several minutes per algorithm per input type was not feasible within the scope of this project. At that size, their quadratic behaviour is well understood theoretically.

6.3 Future Work

There are several ways this study could be extended:

- **Larger inputs:** Testing at $n = 10^6$ or $n = 10^7$ would make the asymptotic differences even more visible and might reveal cache effects that do not show up at smaller sizes.
- **Better pivot for Quick Sort:** Using a random pivot or median-of-three would eliminate the sorted-input worst case and give a fairer comparison with the other efficient algorithms.
- **Hybrid algorithms:** Tim Sort and Intro Sort combine multiple strategies depending on the situation. Testing them alongside the simple algorithms would show how much engineering optimisation helps.
- **Memory usage:** This study only measured time. Tracking peak heap usage would add another dimension, especially relevant for Merge Sort with its $O(n)$ auxiliary space.
- **Parallel variants:** Extending the comparison to parallel Merge Sort or other multi-threaded algorithms would be a natural next step [4, 5].

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [4] D. S. Hirschberg, “Fast parallel sorting algorithms,” *Communications of the ACM*, vol. 21, no. 8, pp. 657–661, 1978.
- [5] N. Satish, M. Harris, and M. Garland, “Designing efficient sorting algorithms for manycore GPUs,” in *Proc. IEEE International Symposium on Parallel & Distributed Processing*, pp. 1–10, 2009.

A Repository and Reproducibility

The full source code for all twelve sorting algorithms and the benchmarking harness, along with input generators for each of the six input types and the raw CSV results used to produce the tables and figure in this paper, are available at:

<https://github.com/mohamadghayada04/sorting-benchmark>

To reproduce the experiments, clone the repository, compile with `javac`, and run `BenchmarkRunner` from the command line. Results are written to `results/timings.csv`. The figure can be regenerated using the Python script at `plot/plot_results.py`.